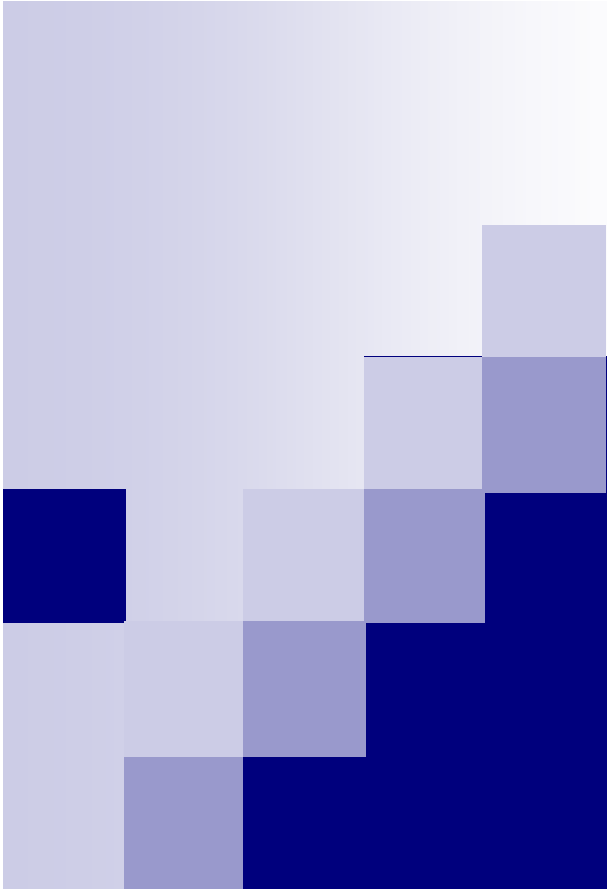




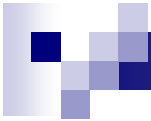
Разработка параллельных программ для систем с распределенной памятью

Технология MPI (Message Passing Interface)



Стандарты MPI

- MPI 1.0 – 1994 год
- MPI 1.1
 - опубликован 12 июня 1995,
 - первая реализация в 2002 году
- MPI 2.0 опубликован 18 июля 1997
- MPI 2.1 – начало сентября 2008
- MPI 2.2 – 4 сентября 2009
- MPI 3.0 – 21 сентября 2012
- MPI 3.1 – 4 июня 2015
- Разработка стандартов - <https://www.mpi-forum.org/>



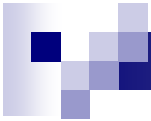
Реализации MPI

■ Открытые

- MPICH - <http://www.mpich.org/>, поддержка MPI-3
- OpenMPI - <https://www.open-mpi.org/>, поддержка MPI-3.1
- MPJ – для Java, <http://www.mpj-express.org/>

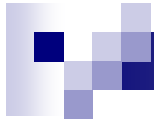
■ Коммерческие

- Intel MPI - <https://software.intel.com/en-us/mpi-library/features>, поддержка MPI-3.1
- HP-MPI - <https://www.hpe.com/us/en/product-catalog/software/high-performance-computing-and-ai-software/pip.hpe-message-passing-interface-mpi.1010144155.html>,
- MPI/PRO - <https://www.runtimecomputing.com/products/mpipro/>, MPI-2



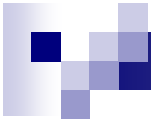
Microsoft MPI

- Если программируете в Visual Studio, то оптимальный вариант. Требуется минимальная настройка
- <https://www.microsoft.com/en-us/download/details.aspx?id=57467>
 - ☐ msmpisdsk.msi – нужные библиотеки
 - ☐ msmpissetup.exe
- В большинстве случаев достаточно будет установить только первый
- Каждый проект, содержащий функции MPI нужно дополнительно настраивать



Основные понятия

- *Параллельная программа* – множество одновременно выполняемых **процессов**
- Процессы могут выполняться на разных процессорах, но на одном процессоре может располагаться несколько процессов
- Количество процессов и число используемых процессоров задается в момент запуска параллельной программы средствами среды исполнения MPI
- Все процессы пронумерованы. Номер процесса – *ранг* процесса



Структура параллельной программы

- Перед использованием функций MPI нужно подключить библиотеку `mpi.h`
 - `#include "mpi.h"`
- Начало параллельной области задаётся функцией
 - `MPI_Init( &argc, &argv);`
- В конце должна быть
 - `MPI_Finalize();`
- Нужно помнить, что у процессов нет доступа к памяти друг друга и всё общение происходит через передачу сообщений.
- Для взаимодействия процессов используются коммутаторы
 - `MPI_COMM_WORLD`



Структура параллельной программы

```
#include "mpi.h"
int main( int argc, char *argv[] )
{
    int proc_num, proc_rank;
    <программный код без использования MPI функций>
    MPI_Init( &argc, &argv);
    MPI_Comm_size( MPI_COMM_WORLD, &proc_num);
    MPI_Comm_rank( MPI_COMM_WORLD, &proc_rank);
    <программный код с использованием MPI функций>
    MPI_Finalize();
    <программный код без использования MPI функций>
}
```



Операции передачи данных

- Парные (point-to-point)
- Коллективные



Передача сообщений

- `int MPI_Send(void *buf, int count,
MPI_Datatype type, int dest,
int tag, MPI_Comm comm),`

где

- ☐ `buf` – адрес буфера памяти, в котором располагаются отправляемые данные
- ☐ `count` – количество элементов данных в сообщении
- ☐ `type` – тип элементов данных в сообщении
- ☐ `dest` – ранг процесса, которому отправляется сообщение
- ☐ `tag` – значение, используемое для идентификации сообщений
- ☐ `comm` – коммуникатор, в рамках которого выполняется передача данных



Типы данных MPI (C/C++)

- MPI_CHAR (char)
- MPI_INT (int)
- MPI_FLOAT (float)
- MPI_DOUBLE (double)
- MPI_CXX_BOOL (bool)
- MPI_CXX_FLOAT_COMPLEX (std::complex<float>)
- и другие... (стр.26-27 стандарта)



Прием сообщений

- `int MPI_Recv(void *buf, int count, MPI_Datatype type, int source, int tag, MPI_Comm comm, MPI_Status *status),`

где

- ☐ `buf` – адрес буфера памяти, в который принимаются данные
- ☐ `count` – размер буфера
- ☐ `type` – тип принимаемых элементов
- ☐ `source` – ранг процесса, от которого принимается сообщение
- ☐ `tag` – значение, используемое для идентификации сообщений
- ☐ `comm` – коммуникатор, в рамках которого выполняется передача данных
- ☐ `status` – указатель на структуру данных с информацией о результате выполнения операции приема данных



Прием сообщений

- `MPI_ANY_SOURCE` – прием сообщения от любого процесса
- `MPI_ANY_TAG` – прием сообщения с любым тегом
- Возвращаемые значения функций MPI (стр.669-671)
 - `MPI_SUCCESS` – успешно выполнена
 - `MPI_ERR_BUFFER` – неправильный указатель на буфер
 - `MPI_ERR_COMM` – неправильный коммуникатор
 - `MPI_ERR_RANK` – неправильный ранг процесса
- `status.MPI_SOURCE` – ранг процесса-отправителя
- `status.MPI_TAG` – тег принятого сообщения
- `MPI_Get_count(MPI_Status * status, MPI_Datatype type, int * count)`



Пример

```
int proc_rank, proc_num;
const int buf_size = 20;
char buf[buf_size];

MPI_Status st;
MPI_Init(&argc,&argv);
MPI_Comm_rank(MPI_COMM_WORLD, &proc_rank);
MPI_Comm_size(MPI_COMM_WORLD, &proc_num);
if (proc_rank == 0) {
    for (int i = 1; i < world_size; i++) {
        MPI_Recv(buf, buf_size, MPI_CHAR, MPI_ANY_SOURCE, MPI_ANY_TAG,
MPI_COMM_WORLD, &st);
        printf("%s\n", buf);
    }
}
else {
    sprintf(buf, "Hello from %d", proc_rank);
    MPI_Send(buf, buf_size, MPI_CHAR, 0, 0, MPI_COMM_WORLD);
}
MPI_Finalize();
```



Компиляция и запуск параллельной программы

- Компиляция

- `mpicxx -cc=g++ name.cpp -o name -O2`

- Запуск

- `mpirun -np N ./name`



Microsoft MPI

- Создать новый проект. Для тестирования правильности можно использовать следующий код

```
#include <mpi.h>

...

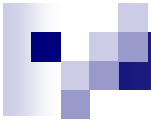
MPI_Init(&argc, &argv);
std::cout << "Hello World!\n";
MPI_Finalize();
```

- Добавить дополнительные каталоги
Проект->Свойства < имя проекта >
 - ☐ C/C++ -> Общие -> Дополнительные каталоги включаемых файлов (...\\Program Files(x86)\\Microsoft SDKs\\MPI\\Include);
 - ☐ Компоновщик -> Общие -> Дополнительные каталоги библиотек (...\\Microsoft SDKs\\MPI\\Lib\\x64);



Microsoft MPI

- Добавить библиотеку
 - Проект->Свойства < имя проекта > -> Компонент -> Ввод -> Дополнительные зависимости (добавить в начало списка `msmpi.lib`);
- Указать компилятор
 - Отладка -> Команда (...\\Program Files\\Microsoft MPI\\Bin\\mpiexec.exe).
- В свойствах проекта установить необходимое число запускаемых процессов
 - Отладка -> аргументы команды (-np 2 `$(TargetPath)`);



Определение времени работы MPI-программы

- `double MPI_Wtime(void)` – возвращает количество секунд, прошедшее с некоторого определенного момента в прошлом

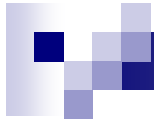
```
double t1, t2, dt;  
t1 = MPI_Wtime();  
...  
t2 = MPI_Wtime();  
dt = t2 - t1;  
std::cout << dt;
```

- `double MPI_Wtick(void)` – точность измерения времени MPI-программы



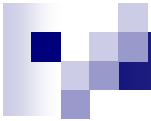
Блокирующее поведение

- Функции `MPI_Send` и `MPI_Recv` являются блокирующими, то есть вызов не возвращает программе управление до тех пор, пока данные не будут скопированы в указанное место (полностью переданы)



Взаимоблокировка потоков

```
if(rank == 0)
{
    MPI_Recv(...1...)
    MPI_Send(... 1 ...)
}
else
{
    MPI_Send(... 0 ...)
    MPI_Recv(...0...)
}
```



Одновременное выполнение передачи и приёма

- `int MPI_Sendrecv(
 void *sbuf, int scount, MPI_Datatype stype,
 int dest, int stag,
 void *rbuf, int rcount, MPI_Datatype rtype,
 int source, int rtag,
 MPI_Comm comm, MPI_Status *status)`
- `int MPI_Sendrecv_replace(
 void *buf, int count, MPI_Datatype type,
 int dest, int stag,
 int source, int rtag,
 MPI_Comm comm, MPI_Status *status)`